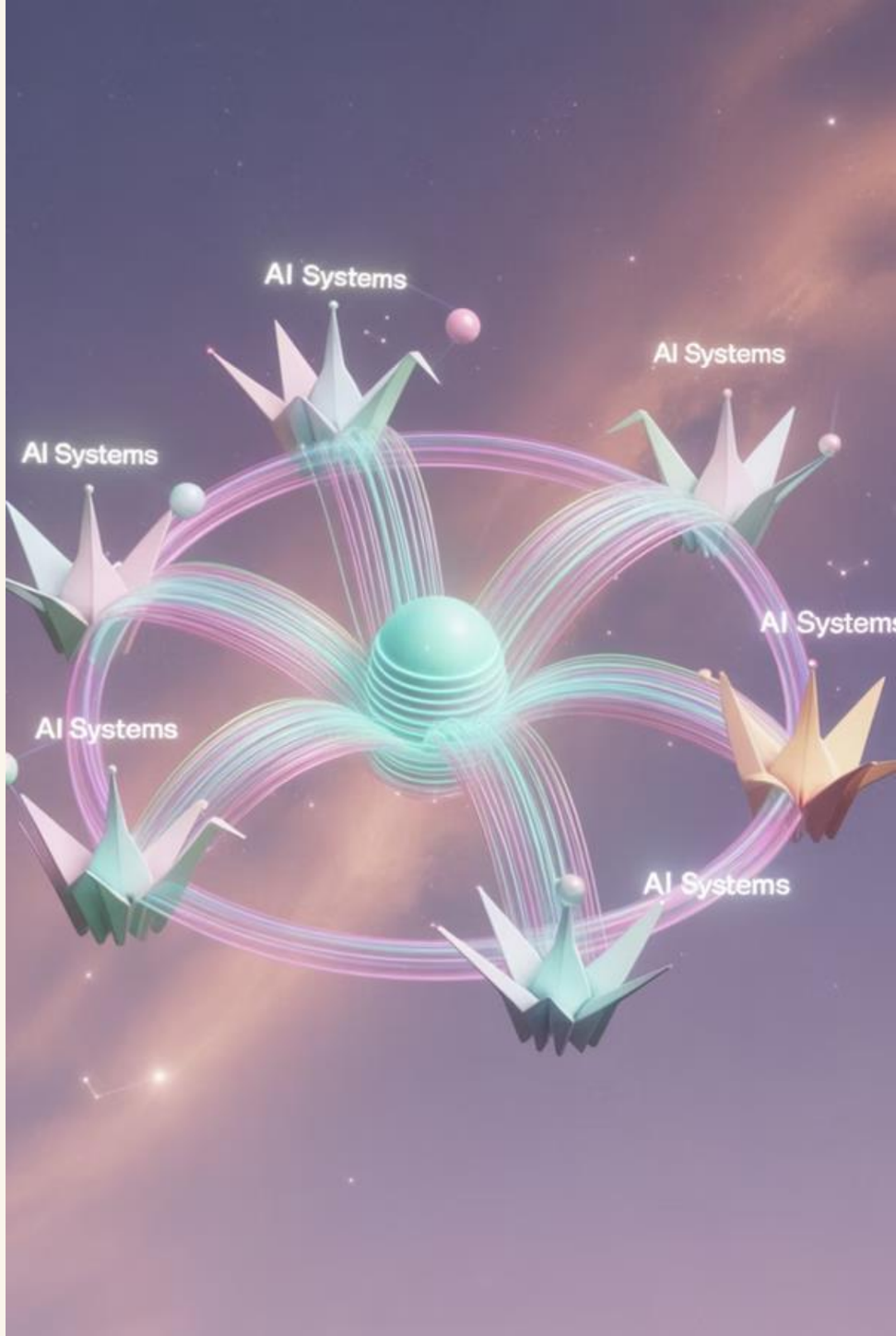
Knowledge Graph-Driven RAG and RAG Evaluation

Enhancing Large Language Models with structured knowledge for more accurate, contextual, and reliable information retrieval



Agenda

Understanding Graph RAG

Foundational concepts bridging LLMs and knowledge graphs

Knowledge Graph Fundamentals

Core components and advantages over unstructured data

Building Knowledge Graphs

Entity and relationship extraction techniques with LLMs

Neo4j Implementation

Storing, querying, and leveraging graph databases

Graph RAG Architecture

End-to-end implementation of knowledge graph-based retrieval

The Evolution of RAG Systems

Basic RAG Limitations

Traditional retrieval-augmented generation systems retrieve raw text passages based on similarity to user queries. While effective for grounding LLM responses in external knowledge, this approach has significant limitations:



Unstructured Text Ambiguity

Processes unstructured text that can be ambiguous



Computational Intensity

Computationally intensive for large document volumes



Misses Contextual Connections

Misses important contextual connections between facts

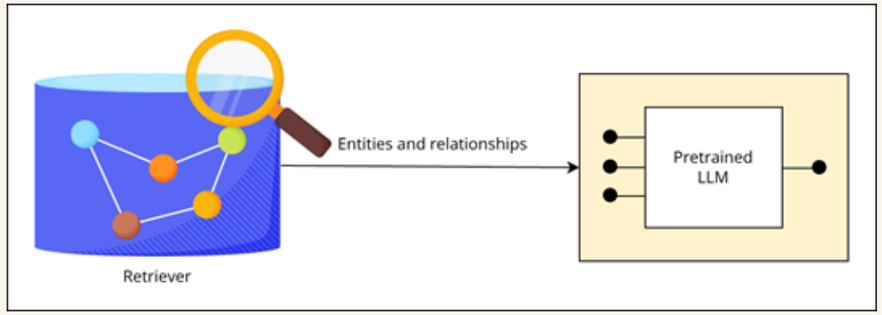


Hallucination Vulnerability

Remains vulnerable to hallucination when faced with ambiguity

Basic RAG retrieves unstructured text to provide context

Knowledge Graph-Based RAG

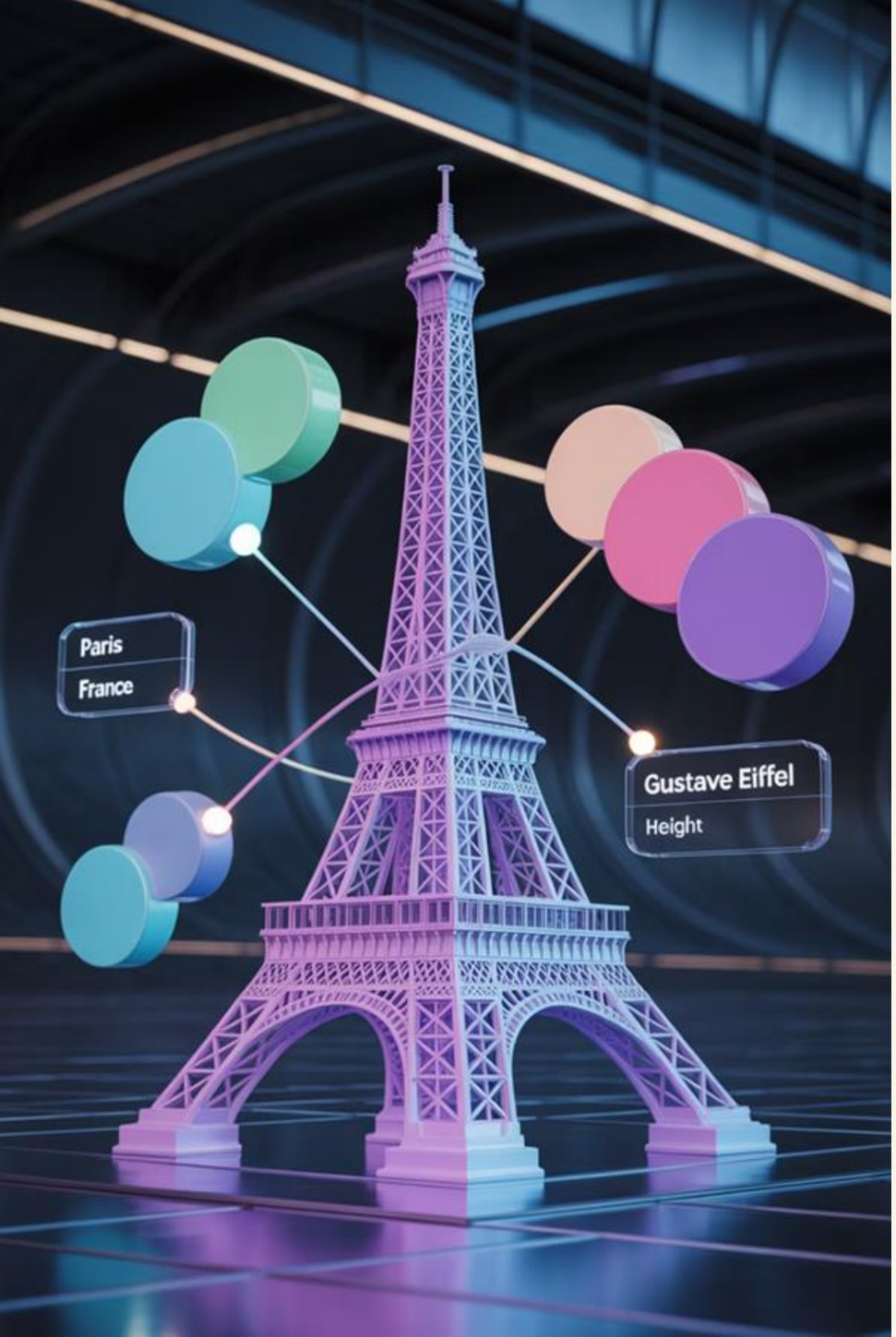


Graph RAG leverages structured relationship data

Graph RAG Advantages

Knowledge graph-based RAG delivers structured, semantically rich data to LLMs by retrieving entities and their relationships rather than raw text:

- Provides clear, structured context with explicit relationships
- Reduces ambiguity through defined entity connections
- Enables more efficient retrieval of relevant information
- Significantly reduces hallucinations through structural constraints



Knowledge Graph Fundamentals

1

Entities

Nodes representing real-world objects or concepts (people, places, organizations, events)

2

Relationships

Edges connecting entities that define specific semantic connections between them

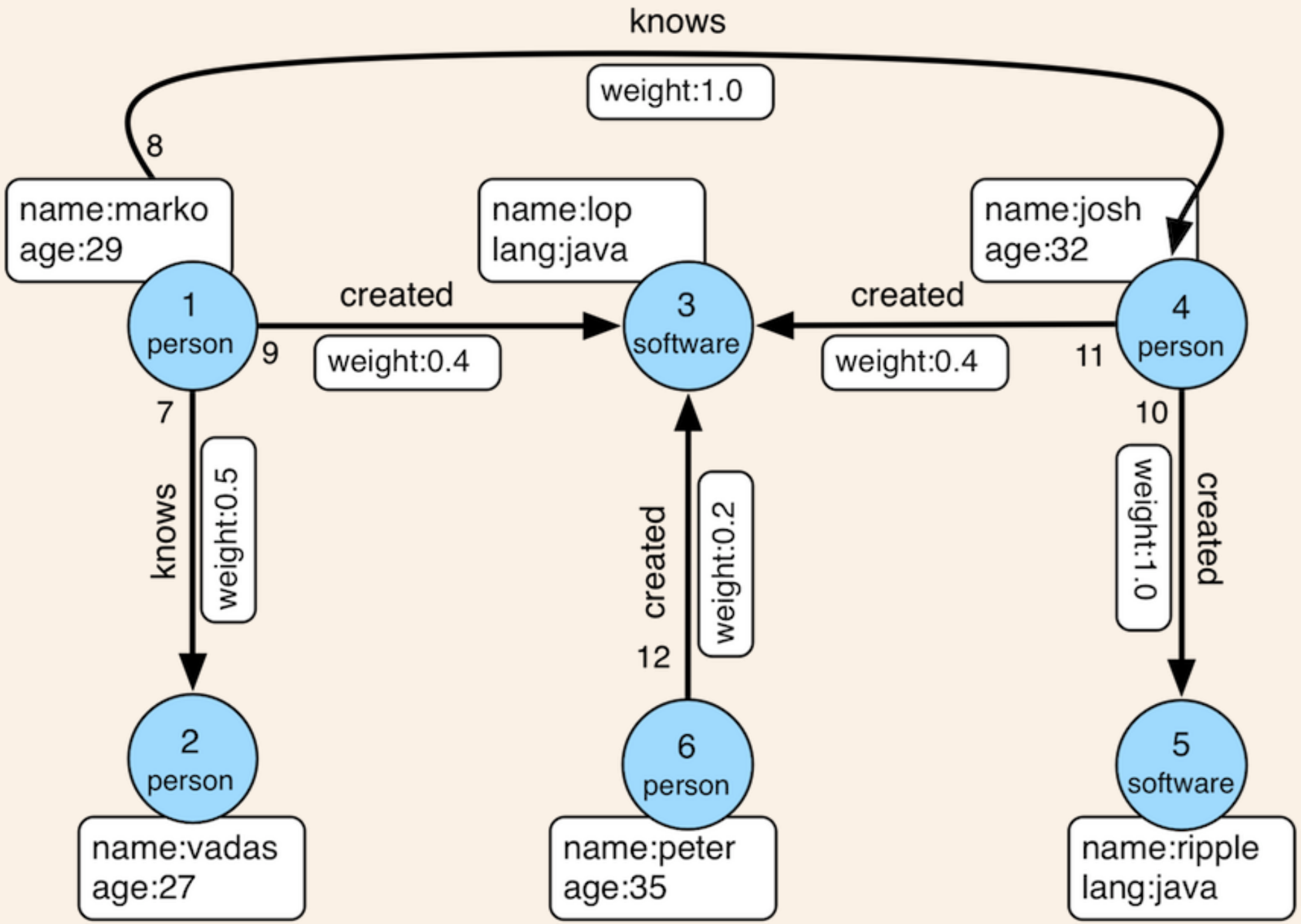
3

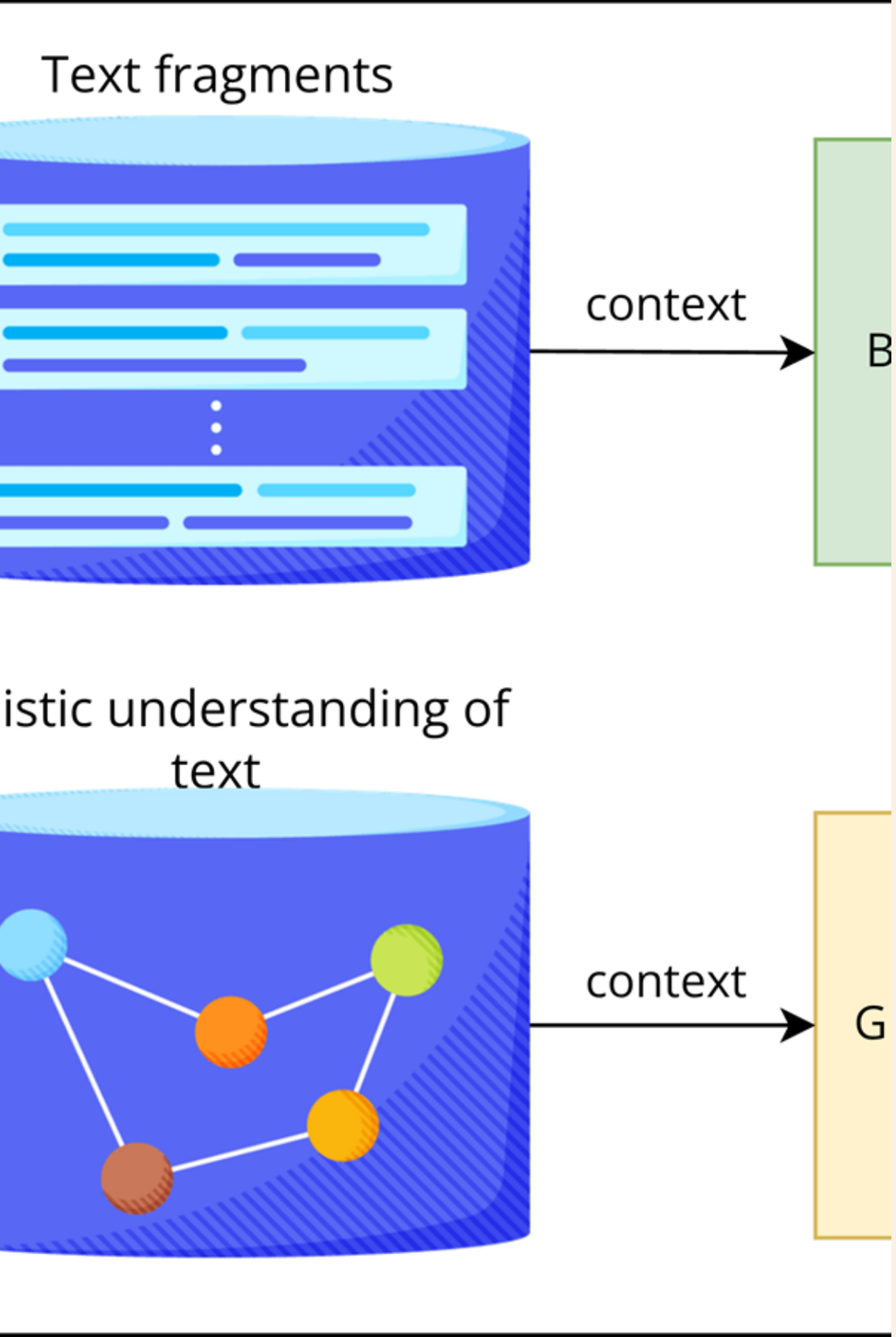
Attributes

Properties providing additional information about entities and relationships

A knowledge graph transforms unstructured text into an interconnected network of facts that machines can navigate and reason with.

Figure: Apache TinkerPop “Modern” Property Graph Example





Retrieval Process Comparison

From Fragmented to Connected Knowledge

Holistic Understanding

Graph RAG considers the full structure of the data, not just isolated text snippets

Reduced Ambiguity

Explicit relationships clarify meaning and minimize potential for hallucination

Comprehensive Responses

Connected information enables more thorough answers by pulling in related facts

Entity and Relationship Extraction

Traditional

Heuristic Methods

Rule-based patterns that identify entities and relationships using predefined templates and linguistic patterns

Statistical ML Models

Supervised learning approaches like CRF, SVM, and LSTM networks trained on labeled datasets to identify entities and relationships

LLM-Enhanced Extraction

Modern approaches leverage LLMs' advanced reasoning capabilities to identify complex entities and relationships with greater accuracy and less training data than traditional methods.



Implementing Entity Extraction with LLMs

Input Text

Raw document or content for knowledge extraction

Chunking

Break large documents into manageable segments for LLM processing

LLM Processing

Prompt the LLM to identify entities and their relationships in structured format

Output

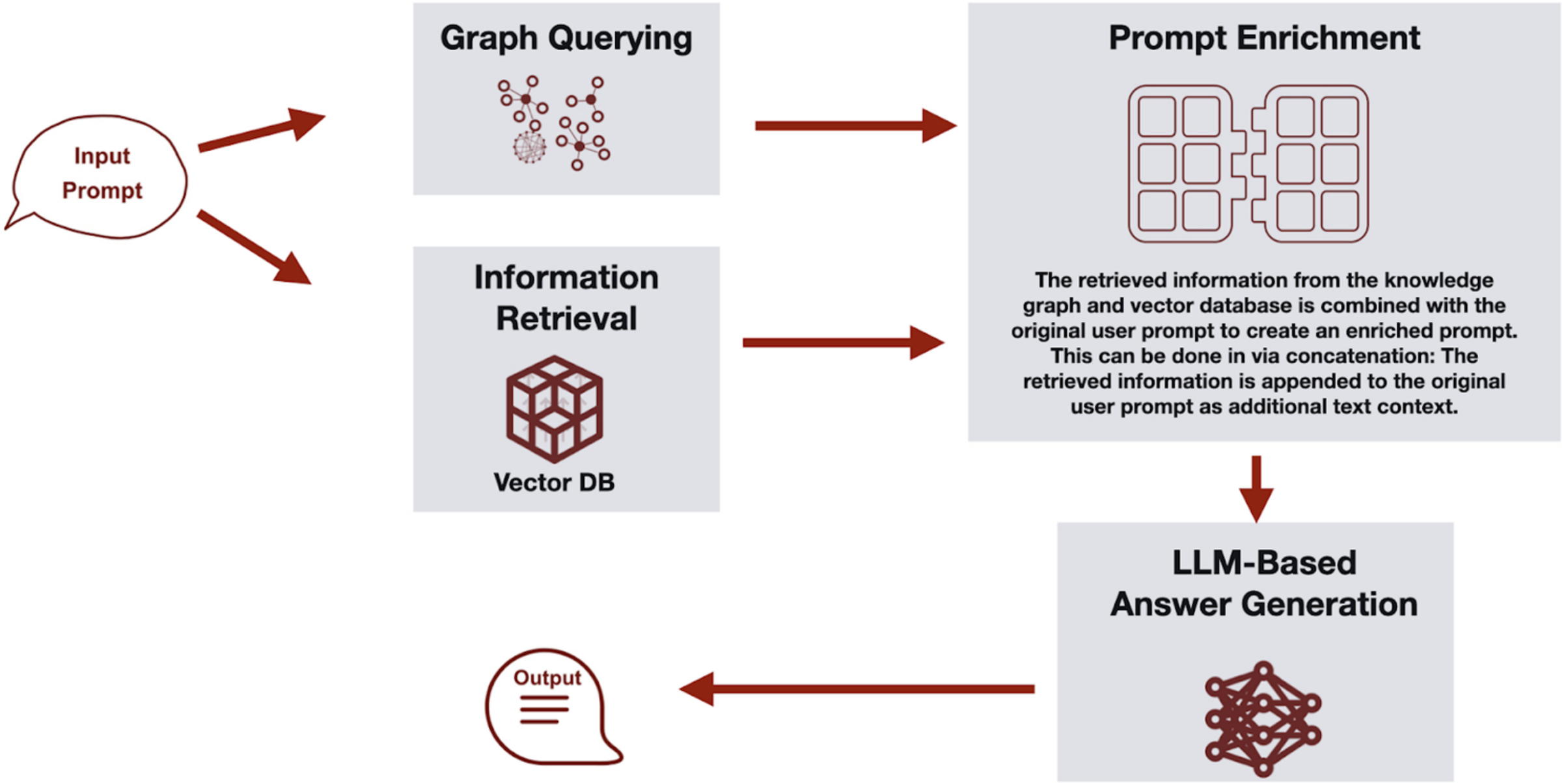
Extract structured entity-relationship tuples for graph construction

Implementation Tip

When extracting entities at scale, implement efficient chunking strategies that preserve contextual information across document segments.

```
# Example Python snippet for entity extraction with OpenAIdef get_entities(query): response = openai.ChatCompletion.create( model="gpt-4", messages=[ {"role": "system", "content": "Extract entities from the following query."}, {"role": "user", "content": query} ] ) # Process the response to extract entities return extracted_entities
```

Knowledge Graph and Vector Database Integration



https://www.puppygraph.com/blog/knowledge-graph-vs-rag

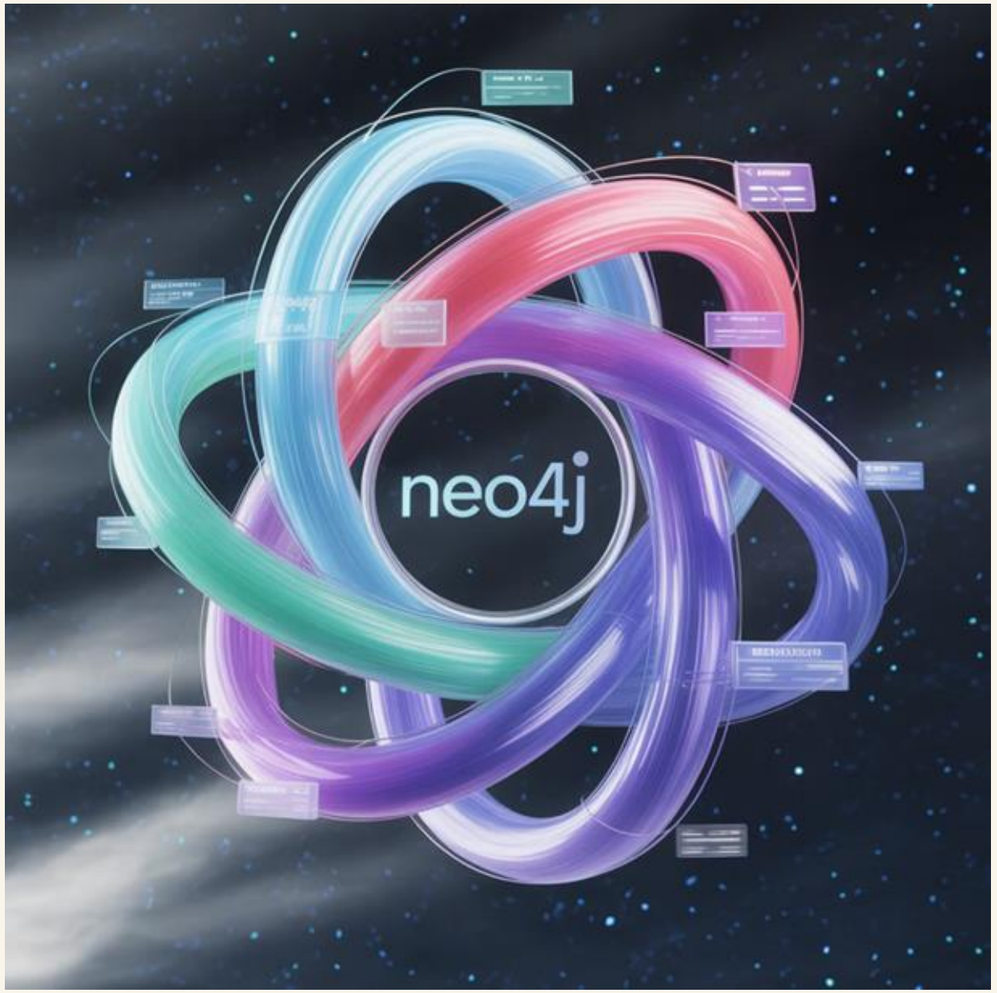
Introduction to Neo4j

What is Neo4j?

Neo4j is a native graph database management system designed specifically for storing, querying, and analyzing graph data structures. Unlike traditional relational databases, Neo4j uses the property graph model with:

- Nodes with labels and properties representing entities
- Relationships with types and properties connecting nodes
- Cypher query language optimized for graph traversal
- Index-free adjacency for efficient relationship navigation

This architecture makes Neo4j ideal for knowledge graphs where relationship-based queries and pattern matching are critical.



Neo4j's property graph model

Working with Neo4j

Basic Cypher Queries for Knowledge Graphs

Creating Nodes

```
CREATE (e:Entity {name: "Eiffel Tower", type: "Landmark"})
```

Creating Relationships

```
MATCH (a:Entity {name: "Eiffel Tower"}), (b:Entity {name: "Paris"})CREATE (a)-[:LOCATED_IN]->(b)
```

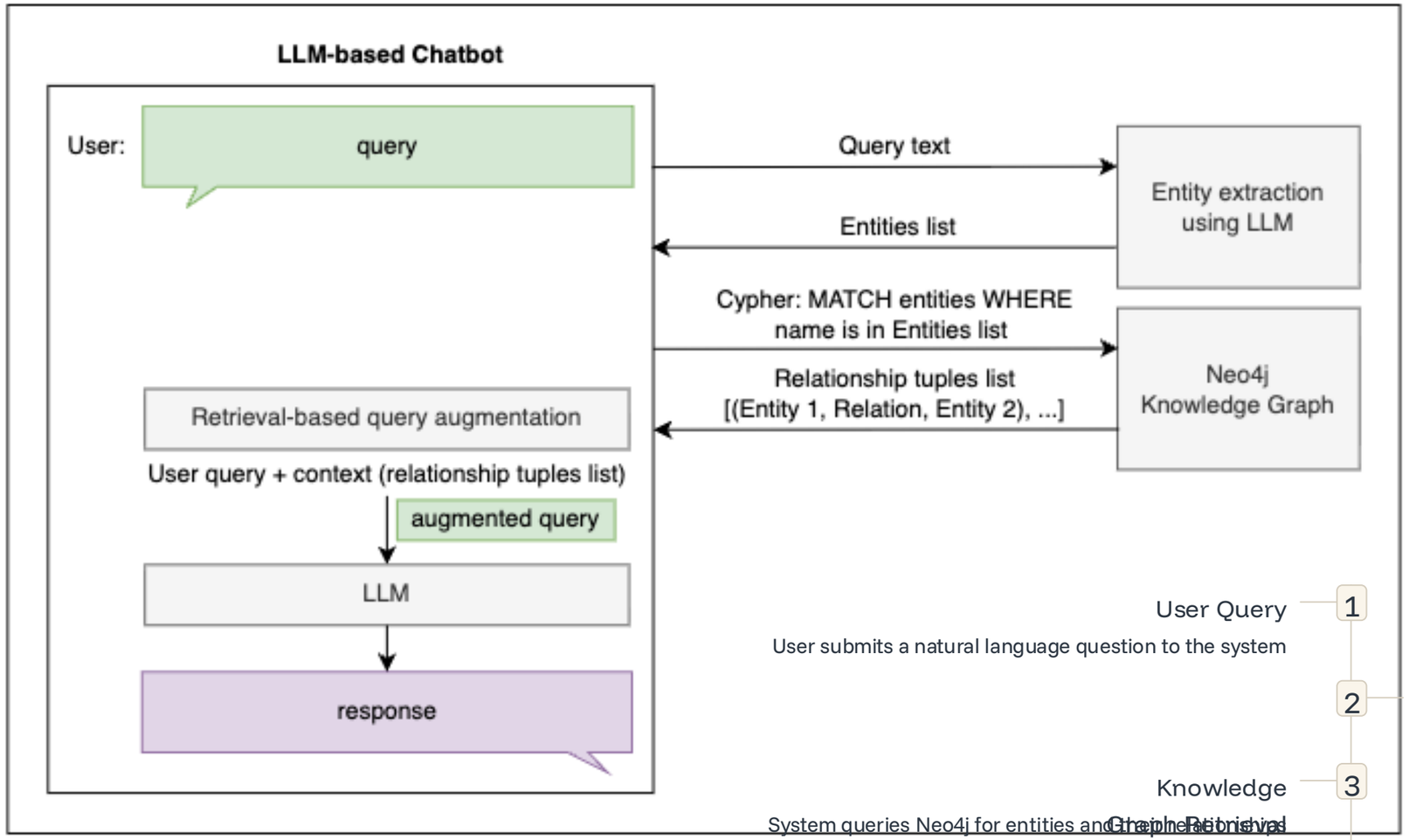
Querying the Graph

```
MATCH (e:Entity)-[r:DESIGNED_BY]->(p:Entity)WHERE e.name = "Eiffel Tower"RETURN e, r, p
```

Pattern Matching

```
MATCH path = (e:Entity {name: "Paris"})-[*1..2]-(related)RETURN path
```

Knowledge Graph-Based RAG Architecture



1 User Query
User submits a natural language question to the system

2 Entity Extraction
LLM extracts relevant entities from the query

3 Knowledge Graph Retrieval
System queries Neo4j for entities and their relationships

4 Context Augmentation
Original query is enhanced with structured knowledge graph data

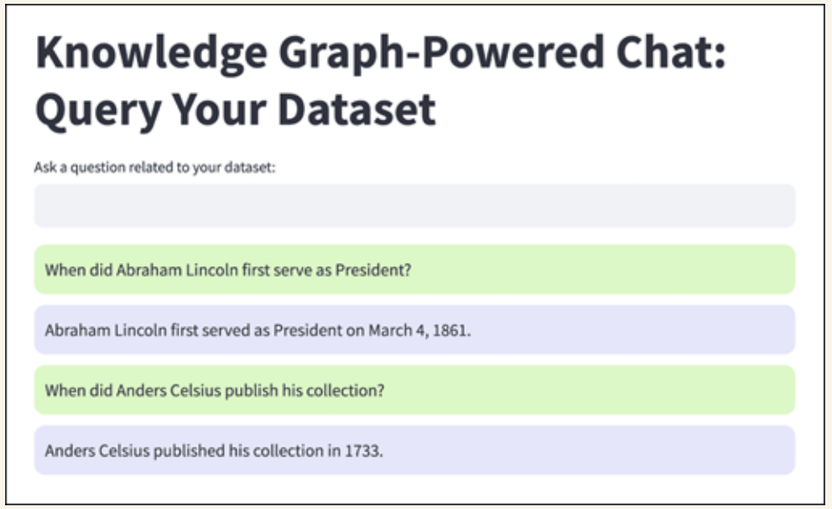
5 Response Generation
LLM generates accurate response based on enriched context

Implementing a Graph RAG-Powered Chatbot

Key Components

1. **Frontend (chat_app.py)**: Streamlit interface for user interaction
2. **Entity Extraction (LLM.py)**: OpenAI API integration for entity recognition
3. **Graph Retrieval (Neo4j.py)**: Neo4j database handler for context retrieval
4. **Response Generation**: LLM-powered answer formulation with graph context

The modular architecture allows for easy replacement of components (e.g., switching LLM providers or database systems) while maintaining the overall workflow.



Example chatbot interface built with Streamlit

Neo4j Integration Code Example

```
from neo4j import GraphDatabase
from typing import Iterable, List, Tuple

class Neo4jHandler:
    def __init__(self, uri: str, username: str, password: str):
        self.driver = GraphDatabase.driver(uri, auth=(username, password))

    def close(self):
        if self.driver:
            self.driver.close()

    def get_entities_and_relationships(self, entity_names: Iterable[str]) -> List[Tuple[str, str, str]]:
        names = list(entity_names)
        if not names:
            return []

        with self.driver.session() as session:
            return session.execute_read(self._retrieve_relationships_bulk, names)

    @staticmethod
    def _retrieve_relationships_bulk(tx, names: List[str]) -> List[Tuple[str, str, str]]:
        query = """
        MATCH (e1:Entity)-[r]->(e2:Entity)
        WHERE e1.name IN $names
        RETURN e1.name AS entity1, type(r) AS relation, e2.name AS entity2
        """
        result = tx.run(query, names=names)
        return [(rec["entity1"], rec["relation"], rec["entity2"]) for rec in result]
```

This Neo4j handler efficiently retrieves entity relationships that match user query entities, providing structured context for LLM response generation.

Example: Graph RAG in Action

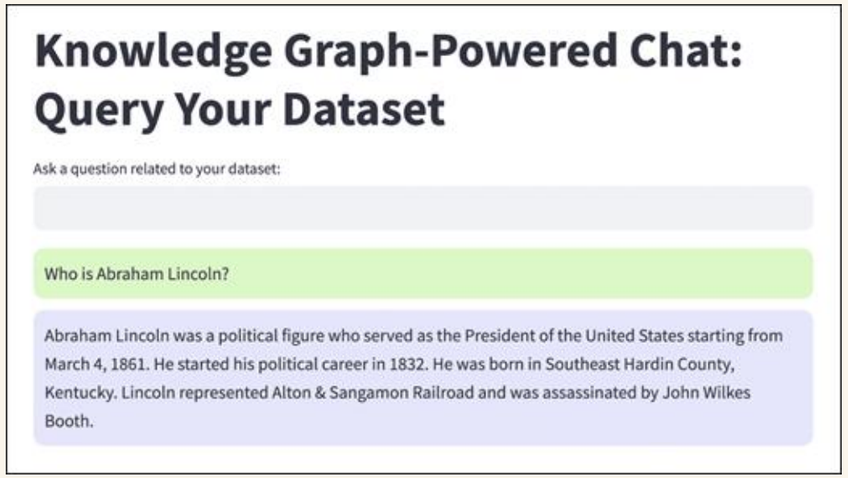
Query: "Who is Abraham Lincoln?"

```
Entities in the query:  ['Abraham Lincoln']
Extracted context from the Knowledge Graph:  [('Abraham Lincoln', 'did represent', 'Alton & Sangamon
Railroad'), ('Abraham Lincoln', 'mother did not die of', 'pneumonia'), ('Abraham Lincoln', 'was born
in', 'Southeast Hardin County, Kentucky'), ('Abraham Lincoln', 'was assassinated by', 'John Wilkes Bo
oth'), ('Abraham Lincoln', 'first served as President on', 'March 4, 1861'), ('Abraham Lincoln', 'sta
rted political career in', '1832')]
```

Retrieved knowledge graph context

Response Generation Process

1. Entity "Abraham_Lincoln" extracted from query
2. Neo4j retrieves all relationships connected to this entity
3. Structured data shows Lincoln was:
    - 16th President of the United States
    - Born in Kentucky
    - Led during the Civil War
    - Issued the Emancipation Proclamation
    - Assassinated by John Wilkes Booth
4. LLM generates comprehensive response based on explicit relationships



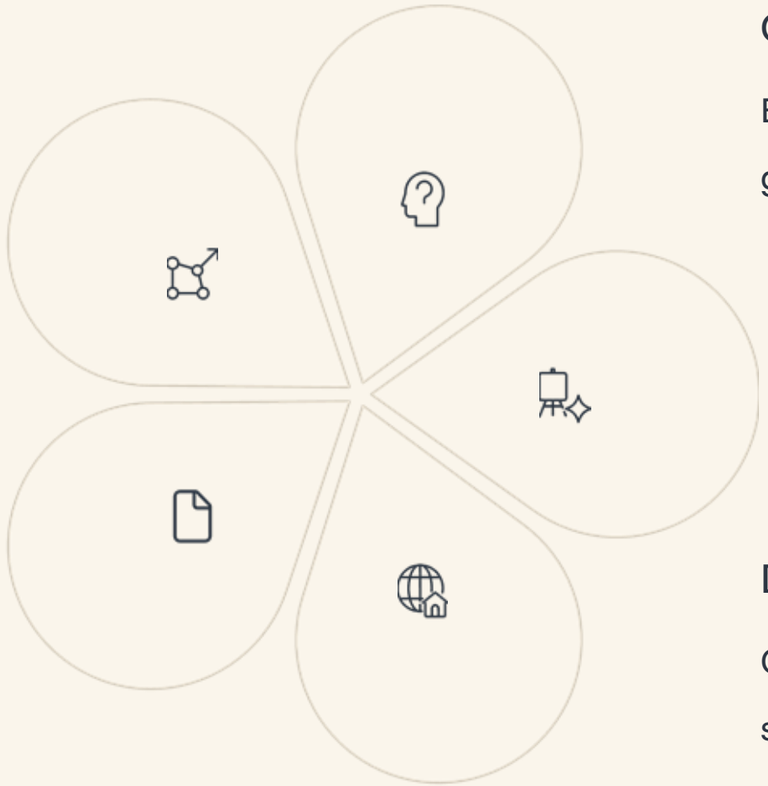
Future Directions for Knowledge Graph RAG

Vector Embeddings

Combine graph structure with semantic similarity through vector embeddings stored in graph nodes

Multimodal Knowledge

Extend knowledge graphs to include connections between text, images, audio, and other modalities



Graph Reasoning

Enable multi-hop inference across knowledge graphs for complex query answering

Dynamic Graphs

Implement continuous learning systems that update knowledge graphs with new information

Domain Specialization

Create purpose-built knowledge graphs for specific industries like healthcare, finance, and legal

Knowledge graph-based RAG represents a significant advancement in making LLMs more accurate, transparent, and capable. By explicitly modeling relationships between entities, we enable more precise information retrieval and reduce hallucinations, creating AI systems that can be truly relied upon for critical applications.

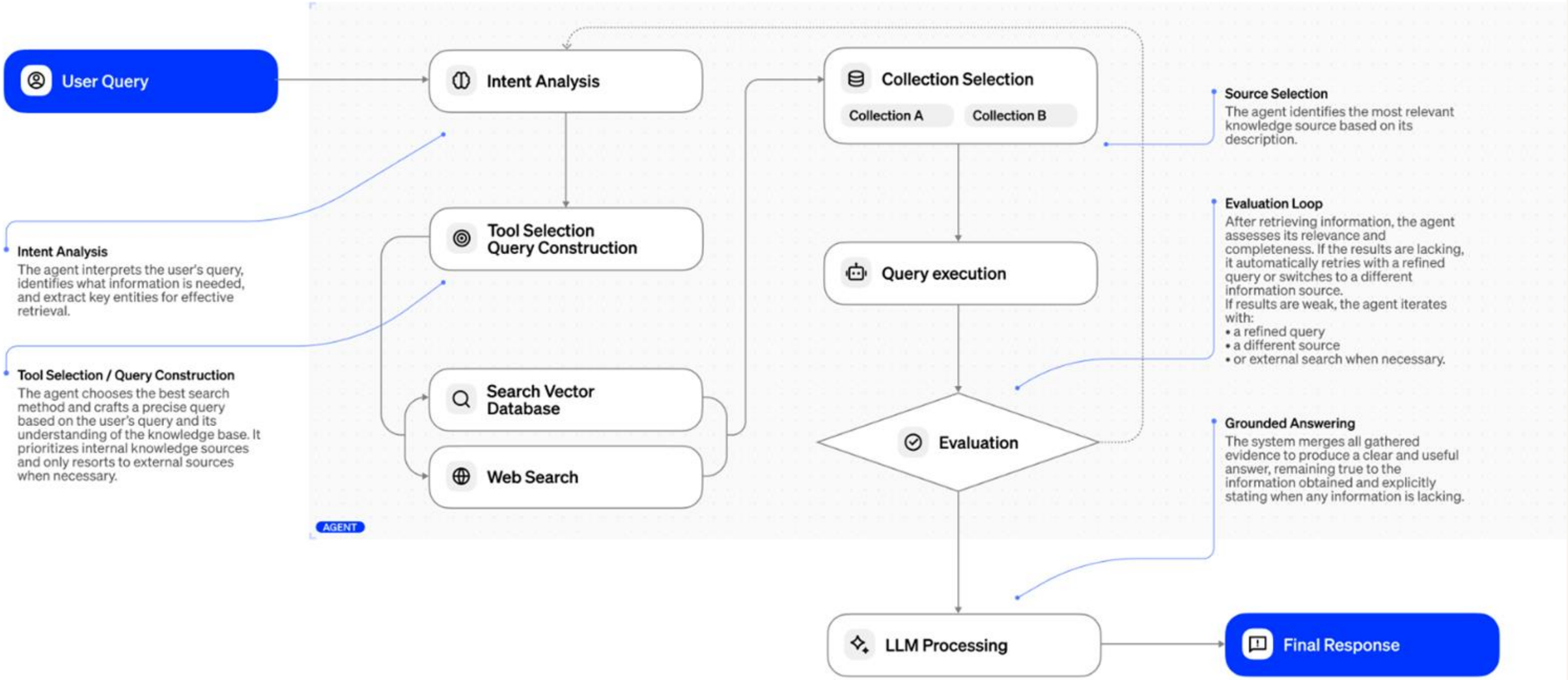
Knowledge Graph vs Traditional RAG

Parameters	RAG (Retrieval-Augmented Generation)	Knowledge Graph Enhanced RAG
Core Approach	Retrieve information from unstructured text and generate responses	Combine Knowledge Graphs with RAG to enable structured
Knowledge Type	Works with unstructured documents or text chunks	Uses both structured and unstructured information
Retrieval Method	Performs vector similarity search using embeddings	Uses hybrid retrieval with graph queries and vector search
Data Representation	Stores text in chunks without explicit relationships	Represents data as nodes and edges
Context Understanding	Limited to text similarity and surface-level meaning	Captures deep semantic and relational context between entities.
Reasoning Capability	Retrieves facts but cannot infer new relationships	Enables multi-hop reasoning and logical inference across connected nodes

https://www.geeksforgeeks.org/artificial-intelligence/knowledge-graphs-for-rag/

Agentic RAG Workflow

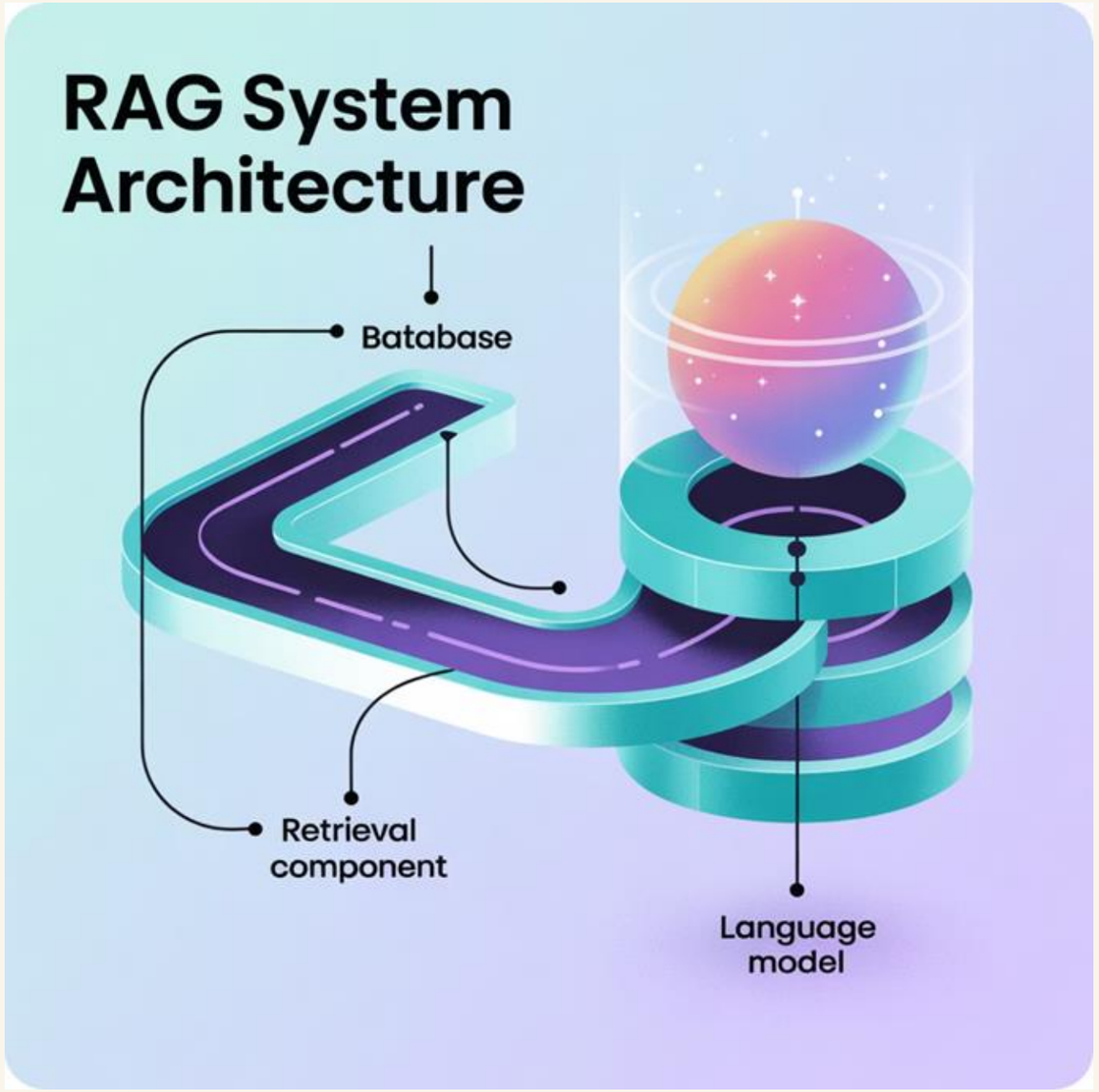
Dify



What is Retrieval-Augmented Generation (RAG)?

RAG is an AI architecture that enhances Large Language Models by connecting them to external knowledge sources. Instead of relying solely on the model's parametric knowledge, RAG systems:

- Retrieve relevant information from databases, documents, or other sources
- Feed this retrieved context to the LLM along with the user query
- Generate responses grounded in specific, up-to-date information
- Reduce hallucinations by providing factual context



Why Evaluate RAG Systems?

Hallucination Risk

Even with external knowledge, RAG systems can still hallucinate if retrieval returns irrelevant documents or if the LLM ignores the provided context.

Pipeline Complexity

RAG combines multiple components (retriever, generator) that must be evaluated both independently and as a complete system to ensure reliability.

Trust Requirements

Enterprise and mission-critical applications demand verifiable performance metrics and guarantees of factual accuracy before deployment.

Without robust evaluation, RAG systems may appear to work while silently producing incorrect or misleading information.

Introducing the RAG Triad: A Comprehensive Evaluation Framework

Developed by TruEra (TruLens), the RAG Triad provides a structured approach to identifying hallucinations and assessing response quality in RAG systems.

Context Relevance

Do retrieved documents match the query?



Groundedness

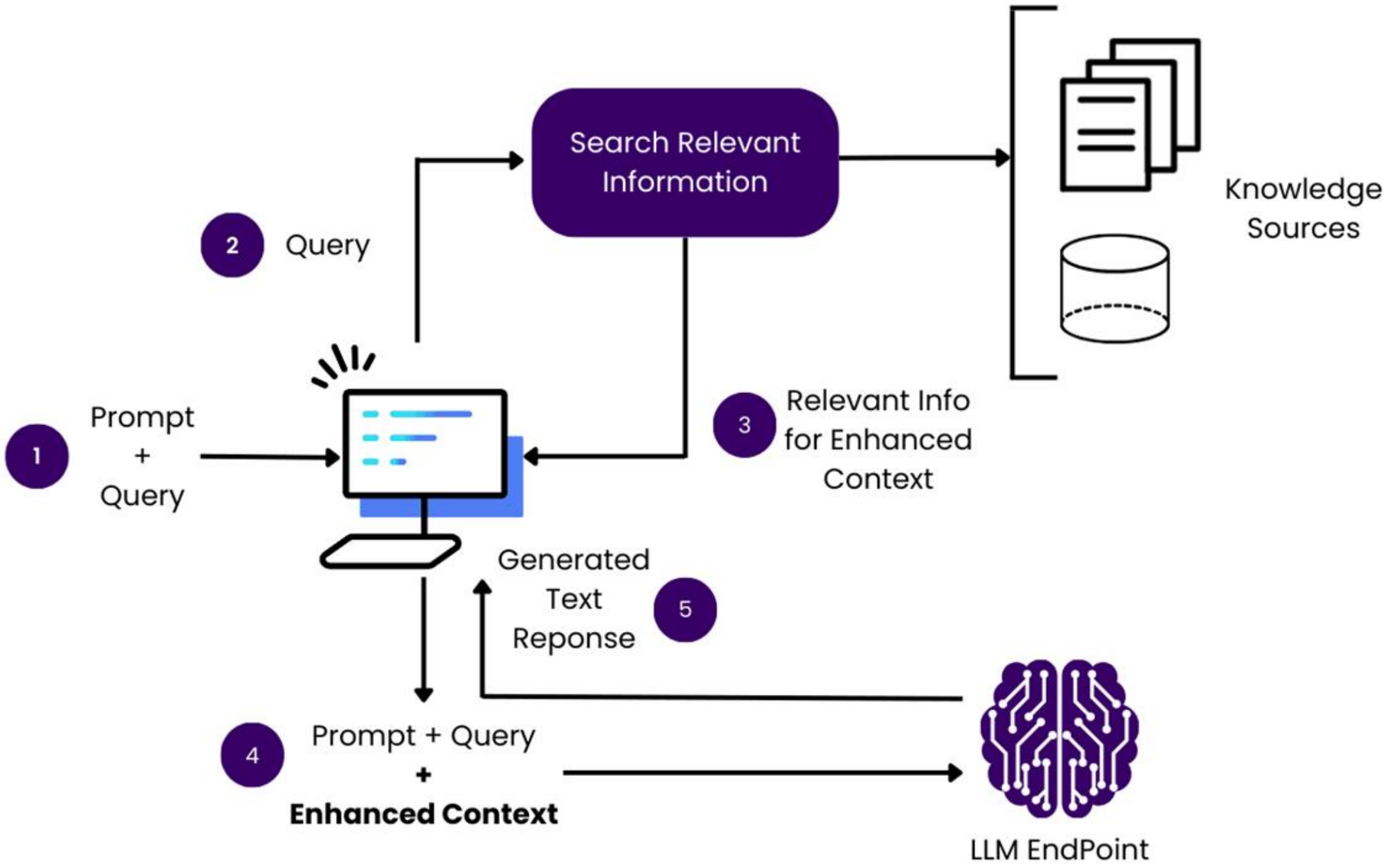
Is the answer supported by retrieved context?

Answer Relevance

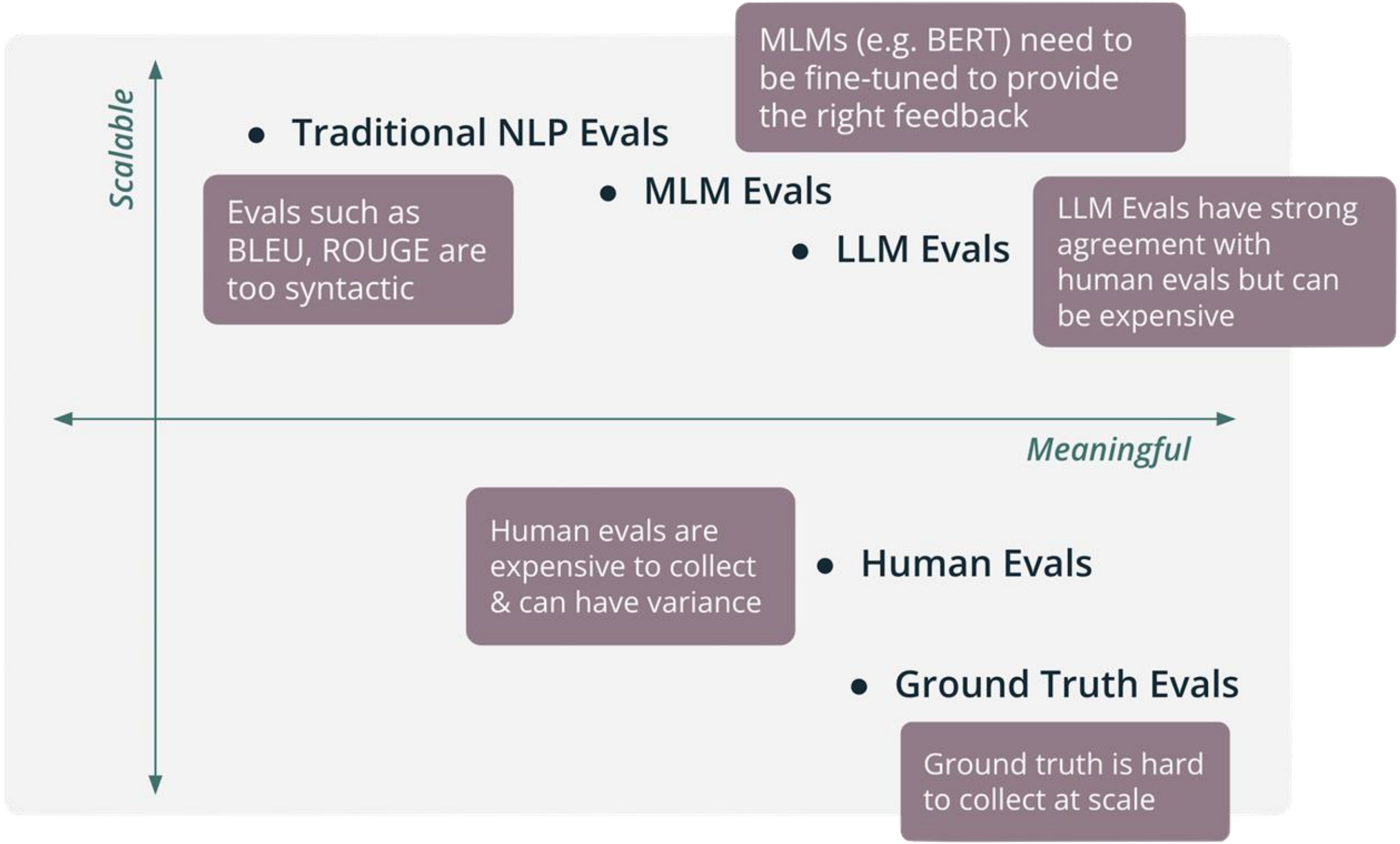
Does the answer address the original question?

When a RAG system passes all three metrics, we can be confident that it's providing factual, well-supported, and helpful responses.

Retrieval-augmented Generation (RAG)

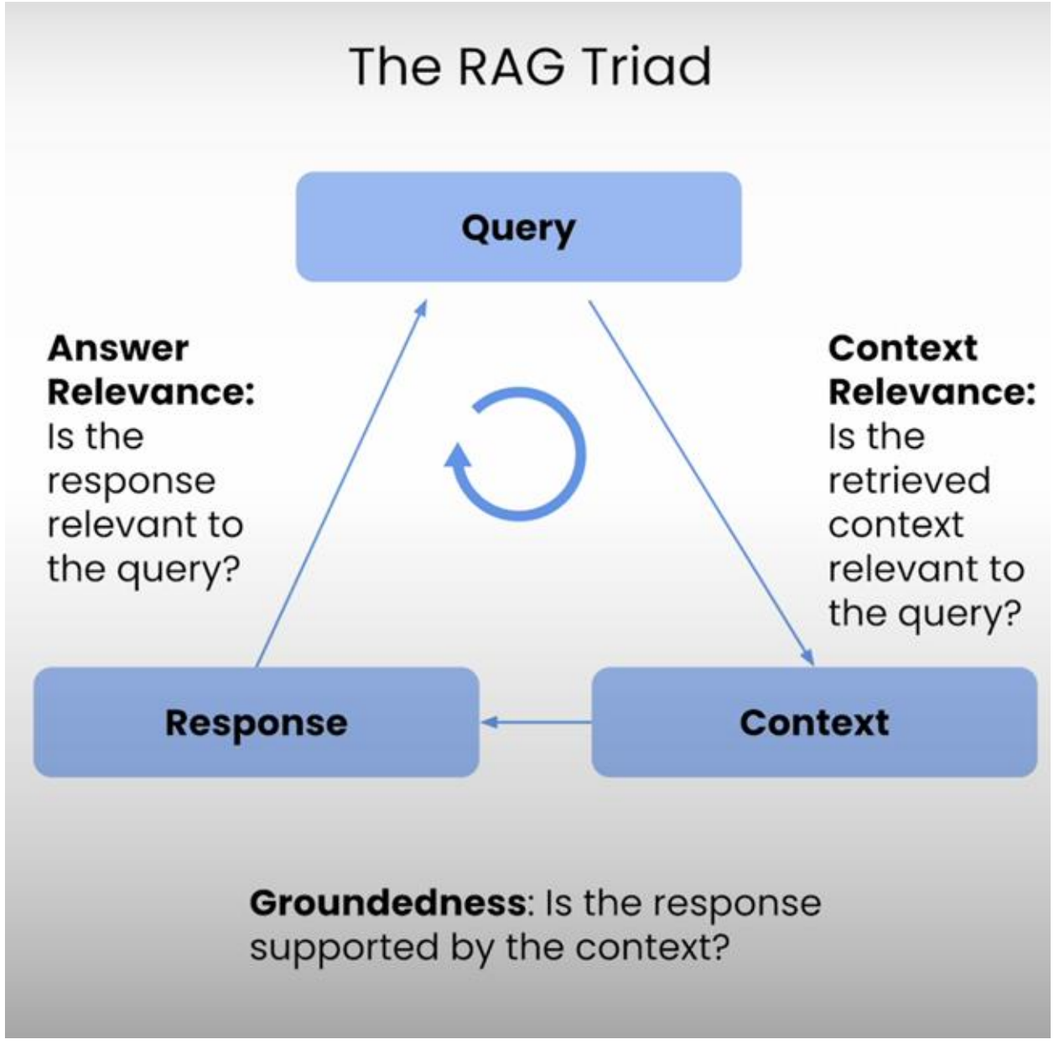


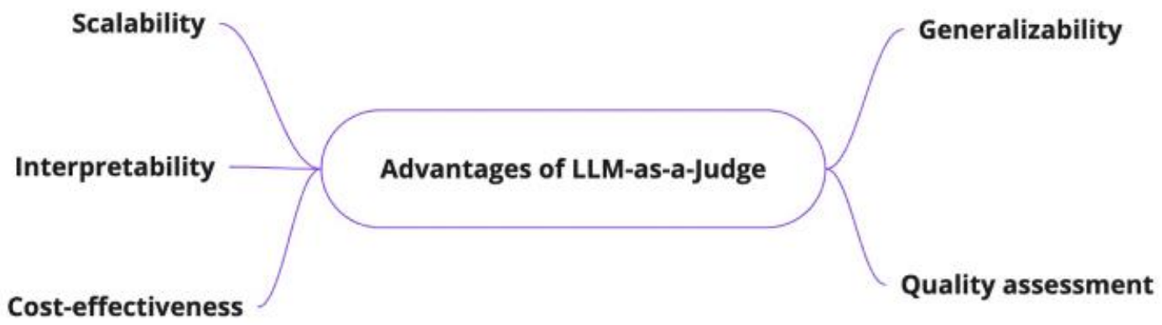
LLM Evaluation

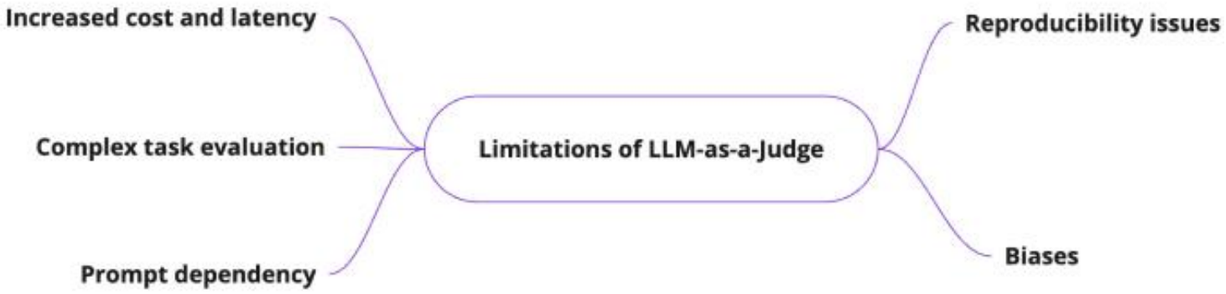


RAG Evaluation (RAG Triad)

- Snowflake TruLens: an open-source tool for quantitative evaluation of LLM models
- Automated Evaluation: Reduces reliance on human annotators, minimizing bias.



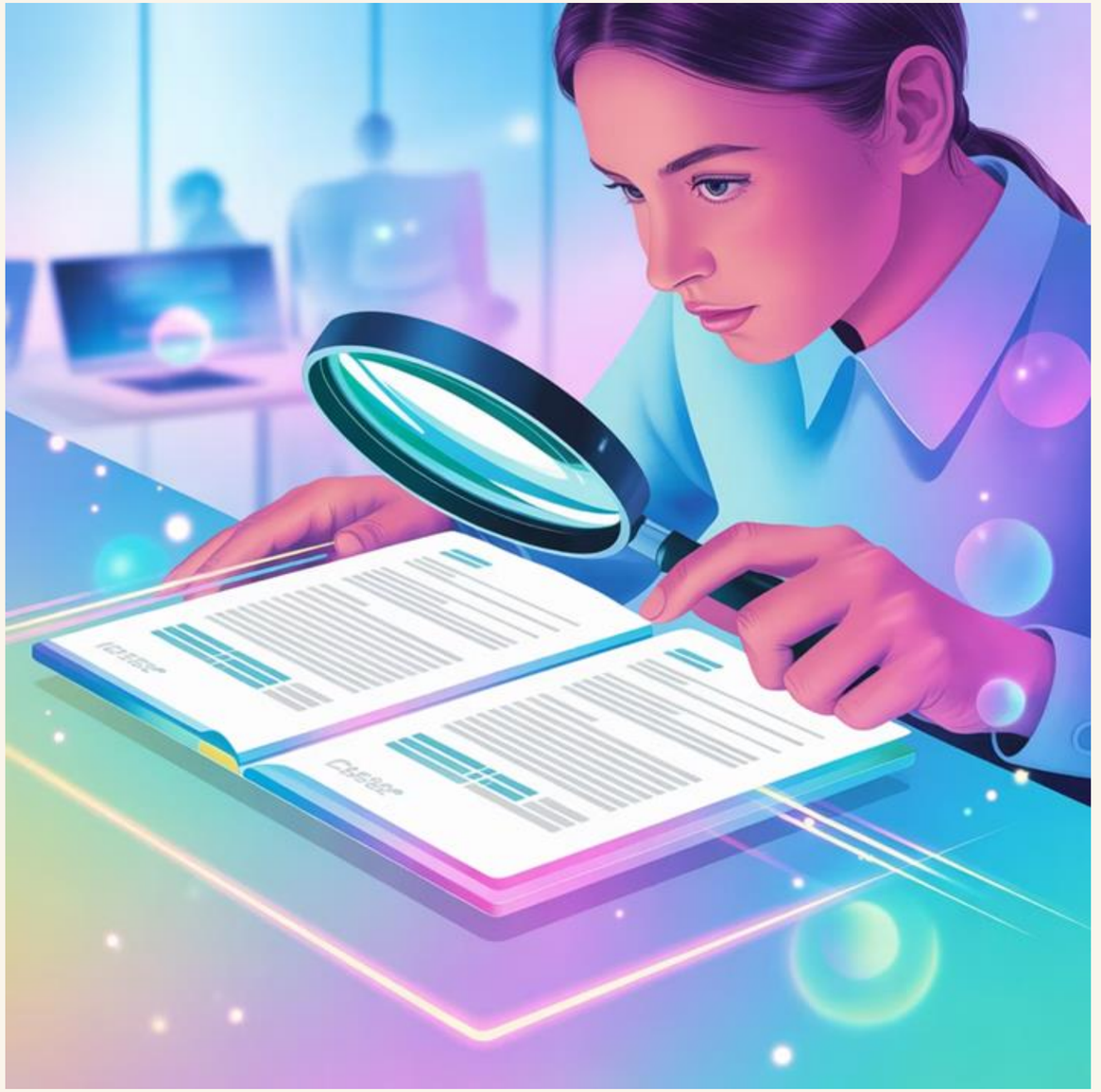




1. Context Relevance: Is the Retrieved Context Relevant?

Context relevance evaluates how well the retrieval component of RAG performs. It measures whether the documents or chunks fetched from the knowledge base match what the user is asking about.

- Typically scored by LLMs or embedding similarity on a 0-10 scale
- Critical first step - irrelevant context leads to hallucinations
- Helps diagnose retrieval failures vs. generation failures



Example: For the query "Who introduced relativity?", documents about Einstein's work would

2. Groundedness: Is the Answer Supported by the Retrieved Context?

1

Break Answer into Claims

Decompose the generated answer into individual factual assertions that can be verified independently.

2

Find Supporting Evidence

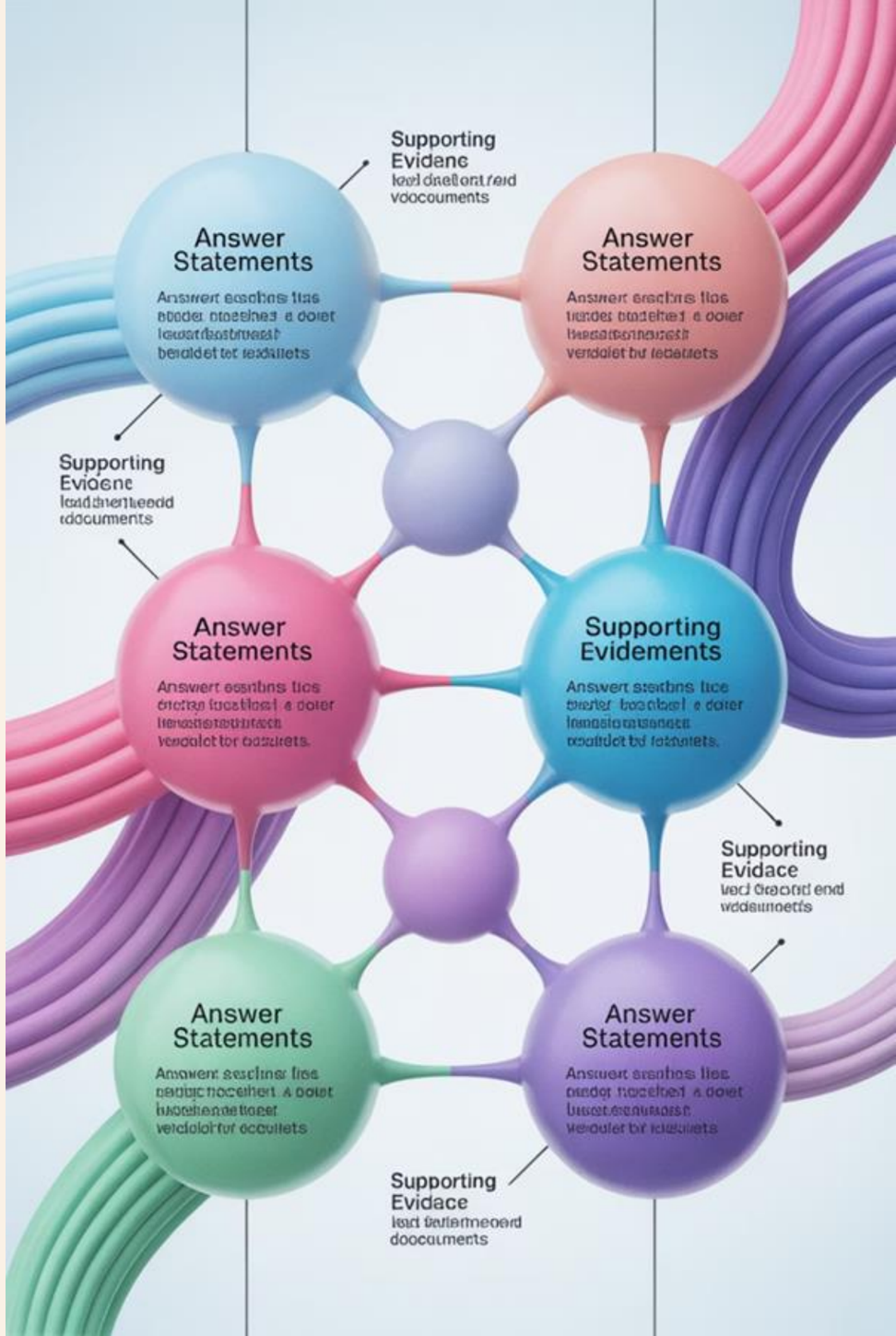
For each claim, search the retrieved context to find corroborating information.

3

Score Support Level

Determine if each claim is fully supported, partially supported, or unsupported by the context.

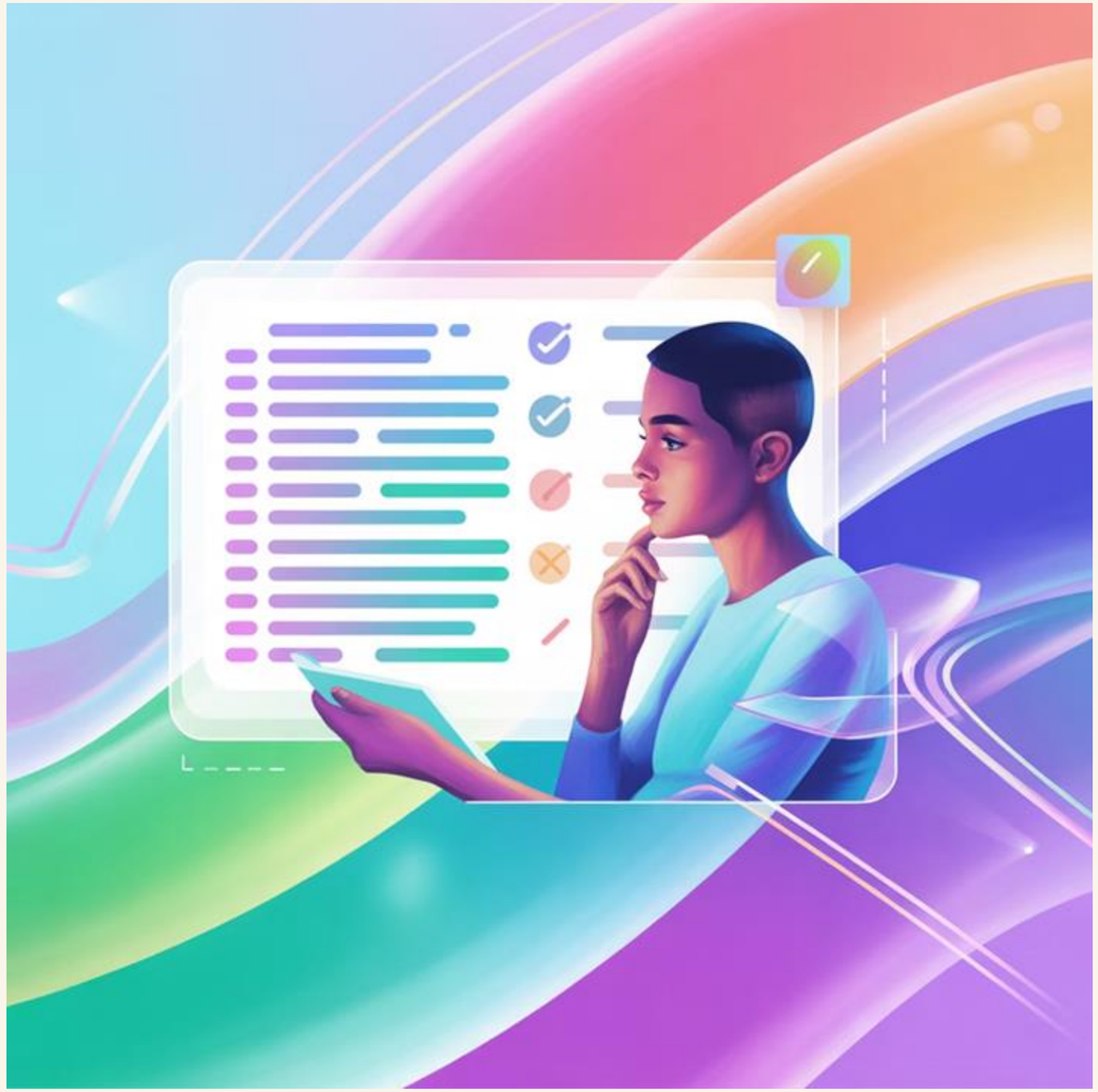
Groundedness catches cases where the LLM fabricates information or "hallucinates" facts not present in the retrieved documents, even when those documents are relevant.



3. Answer Relevance: Does the Answer Address the Original Question?

Answer relevance assesses whether the final response actually addresses what the user asked, regardless of its factual accuracy or groundedness in context.

- Prevents technically correct but unhelpful responses
- Ensures LLM doesn't ignore the query to focus on tangential information
- Typically scored using semantic similarity or LLM-based evaluators



This metric closes the loop by confirming that even with relevant context and well-grounded generation,

Putting It All Together: The RAG Triad Workflow

Context Retrieval & Evaluation

The system retrieves documents from the knowledge base in response to a query. We evaluate whether these documents contain information relevant to the question.

Answer Generation & Groundedness Check

The LLM generates an answer based on the retrieved context. We verify that all claims in the answer are supported by information in the context.

Answer Relevance Assessment

Finally, we compare the generated answer to the original query to ensure it properly addresses the user's information need.

Only when a RAG system passes all three evaluations can we be confident it's providing factually accurate, well-grounded, and truly helpful responses to users.

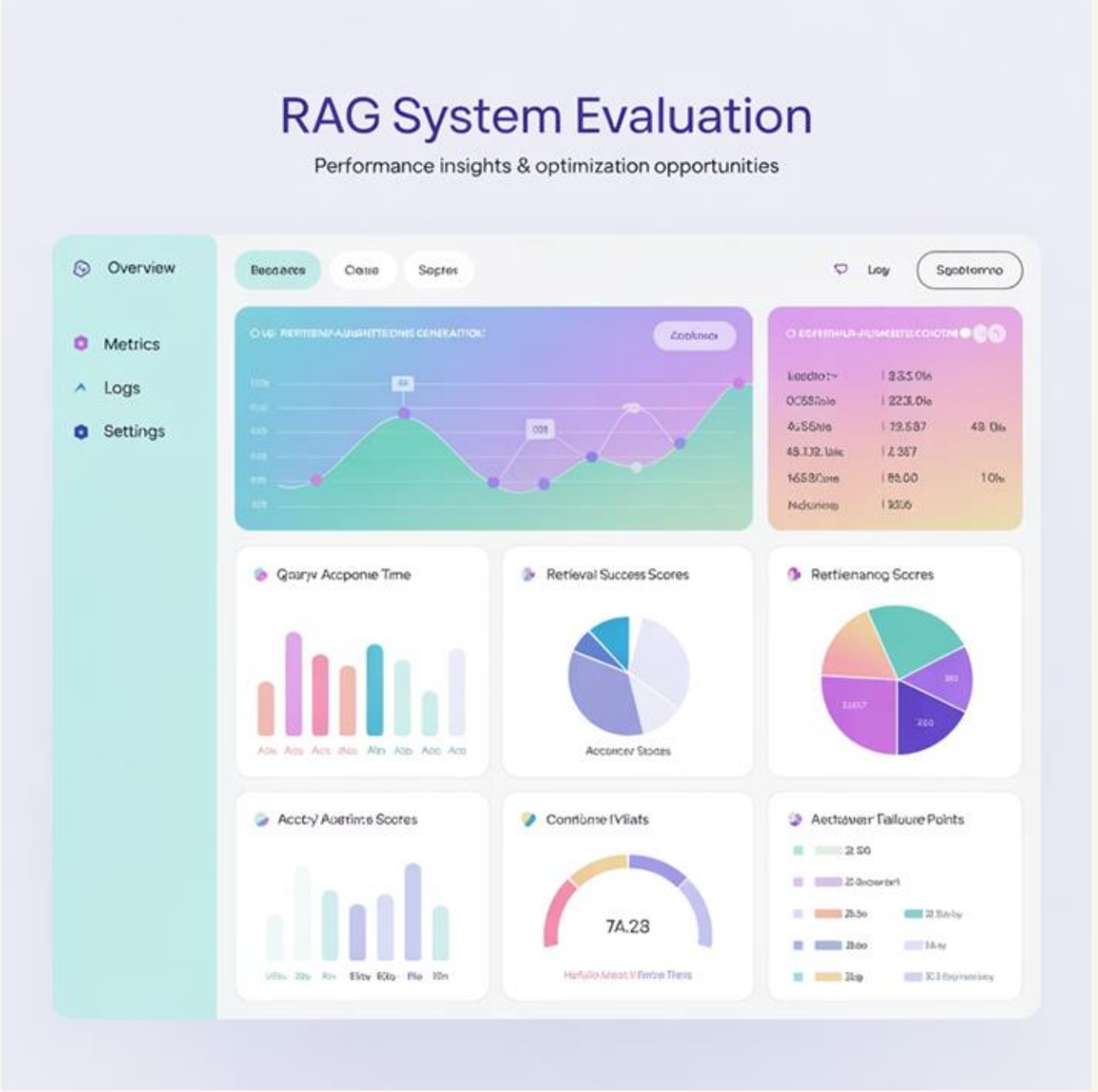
Practical Evaluation with RAG Triad & Tools

Evaluation Frameworks

- TruLens: Open-source tool for automated RAG evaluation
- RAGAS: Specialized framework for assessing RAG systems
- Custom evaluation pipelines using LLM-as-a-judge approaches

Key Metrics to Track

- Precision and recall for context relevance
- Citation accuracy and faithfulness for groundedness
- Semantic similarity scores for answer relevance



Regular evaluation using representative test sets allows for continuous improvement of both retrieval mechanisms and prompt engineering for the generation component.



Conclusion: Why RAG Triad Matters for Responsible AI

Enhanced Trust

By systematically evaluating RAG systems using the triad approach, organizations can confidently deploy AI solutions with minimal hallucination risk.

Targeted Improvements

The triad identifies specific weaknesses in retrieval vs. generation, allowing engineers to focus optimization efforts where they matter most.

Responsible AI

As LLMs become more prevalent in critical applications, robust evaluation frameworks like RAG Triad become essential safeguards against misinformation.

Next steps: Integrate RAG Triad evaluation into your development workflow, establish baselines for your specific use cases, and implement continuous monitoring to ensure sustained performance as your knowledge base and user queries evolve.